Creating and Managing Database with PGADMIN

1. PostgreSQL vs PgAdmin

- PostgreSQL – Actual Database
- PgAdmin – A graphical/web-based UI used to manage PostgreSQL

```
pgAdmin
   ↓
PostgreSQL
   ↓
Databases / Tables / Data
```

2. Start the containers

- First make sure the Docker Compose services are running
- Run them in detached mode

Bash

```
docker compose up -d
```

- This starts the PostgreSQL and pgAdmin containers in the background.

3. Open PgAdmin

Open your browser and go to:

```
http://localhost:5050
```

- pgAdmin will open in the browser.
- The lecture shows it redirecting to `/browser`.

4. Set the PgAdmin Master Password

- pgAdmin asks you to set a **master password**
- Set a password of your choice
- This password is for accessing the pgAdmin interface
- Important – this is the pgAdmin master password. It is separate from the PostgreSQL database user's password.

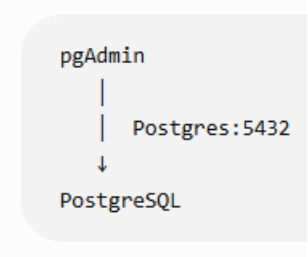
5. Register the PostgreSQL Server

- Initially pgAdmin doesn’t have any server configured
- Go to



6. Go to the Connection Tab

- Now open the **Connection Tab**
- We need to provide the **PostgreSQL** connection details
- **Hostname / Address** use – **postgres**
- Why ?
  - **Postgre** is the **connection/service** name defined in **Docker Compose**
  - Since PostgreSQL and PgAdmin are connected to the same **Docker Network, PgAdmin** can use **container name** as the **host name**



7. PostgreSQL Port

- Use 5432
- PostgreSQL is running on port 5432
- So the connection is Postgre:5432

8. PostgreSQL username

- Enter the username configured in the Docker compose file
- Enter the PostgreSQL password that was configured in Docker Compose file
- Important Distinction

There are now **two passwords** to keep straight:

```
pgAdmin Master Password
        ↓
Used to access pgAdmin UI

PostgreSQL Password
        ↓
Used to connect to PostgreSQL database
```

9. Test / Save the Connection

- After entering
  - Host
  - Port
  - Username
  - Password
- Click Save
- If the credentials are incorrect, pgAdmin will show a **connection error**
- Once the correct credentials are provided and connection is successful then the PostgreSQL server will appear under **Servers.**

10. Find the Database

Expand:

```
Servers
   ↓
Postgres
   ↓
Databases
```

You will see the databases available in PostgreSQL.

11. Create the Application DataBase

Our Spring Boot application expects a database named:

```
ecom_db
```

So we need to create this database in PostgreSQL.

In pgAdmin:

```
Databases
   ↓
Right Click
   ↓
Create
   ↓
Database
```

Enter:

```
ecom_db
```

Then save.

12. Database Name must match Application Configuration

Earlier, we configured the Spring Boot application with something like:

```
jdbc:postgresql://localhost:5432/ecom_db
```

Therefore, PostgreSQL should contain:

```
ecom_db
```

The names must match:

```
Spring Boot
     |
     | jdbc:postgresql://.../ecom_db
     ↓
PostgreSQL
     |
     └── ecom_db
```

If your application configuration says:

```
my_database
```

then the PostgreSQL database should be named:

```
my_database
```

13. Key Take Away

- pgAdmin is the UI through which we connect to and manage PostgreSQL
- PostgreSQL **host name** is the Docker service/container name because both the **containers** are **connected** through the **same Docker Network**
- The database name must match **match** the database configured in SpringBoot application